

Вот подробный конспект семинара по Модулям 3-4. Структура адаптирована под конвертацию в `.ipynb` : теория чередуется с кодовыми ячейками, каждое задание имеет пример решения.

Семинар: Модули 3-4. Сетевые протоколы и FastAPI

Часть 1. Сетевое программирование и протоколы

1.1. Модель OSI и стек TCP/IP

Протокол — это набор правил обмена сообщениями. Чтобы управлять сложностью, сеть разделена на уровни абстракции.

Модель OSI (7 уровней):

Уровень	Название	Единица данных	Примеры
7	Прикладной	Данные	HTTP, FTP, SMTP, DNS
6	Представления	Данные	TLS, ASN.1
5	Сеансовый	Данные	NetBIOS, RPC
4	Транспортный	Сегмент	TCP, UDP
3	Сетевой	Пакет	IP, ICMP, OSPF
2	Канальный	Кадр	Ethernet, Wi-Fi, ARP
1	Физический	Бит	Витая пара, оптоволокно

Стек TCP/IP (4 уровня) — практический стандарт Интернета:

TCP/IP	Соответствие OSI	Протоколы
Прикладной	5-7 OSI	HTTP, HTTPS, DNS, SSH
Транспортный	4 OSI	TCP, UDP

TCP/IP	Соответствие OSI	Протоколы
Межсетевой	3 OSI	IP, ICMP
Канальный	1-2 OSI	Ethernet, Wi-Fi, ARP

Когда FastAPI получает HTTP-запрос, данные проходят все уровни снизу вверх: физический -> канальный -> сетевой (IP) -> транспортный (TCP, порт 443) -> прикладной (HTTP).

1.2. TCP: надежность в ненадежном мире

IP — без установления соединения и без гарантий доставки. TCP добавляет:

1. **Three-way handshake** — установление соединения.
2. **Sequence numbers + ACK** — нумерация байтов и подтверждения.
3. **Повторная передача** при потере.
4. **Flow control** (окно получателя `rwnd`) — не засыпать медленного получателя.
5. **Congestion control** (окно перегрузки `cwnd`) — не коллапсировать сеть.

Three-way handshake:

```
In [1]:
Клиент                                Сервер
|                                     |
|----- SYN -----> | (seq = x)
|<--- SYN + ACK ----- | (seq = y, ack = x+1)
|----- ACK -----> | (ack = y+1)
[Соединение установлено]
```

AIMD (Additive Increase, Multiplicative Decrease):

- Slow Start: `cwnd` растет экспоненциально.
- Congestion Avoidance: `cwnd` растет линейно.
- Потеря пакета: `cwnd` делится пополам.

Реальное окно отправки: `min(rwnd, cwnd)`.

Порты:

- 0-1023: Well-known (HTTP: 80, HTTPS: 443, SSH: 22).
- 1024-49151: Registered (PostgreSQL: 5432, Redis: 6379).

- 49152-65535: Dynamic/Private.

1.3. HTTP как текстовый протокол поверх TCP

```
In [2]:
Клиент                                     Сервер
|                                           |
|===== TCP handshake (SYN-SYN/ACK-ACK) =====|
|                                           |
|--- GET /api/users HTTP/1.1 ----->|
|   Host: api.example.com                 |
|   Accept: application/json              |
|                                           |
|<-- HTTP/1.1 200 OK -----|
|   Content-Type: application/json        |
|   Content-Length: 47                   |
|                                           |
|   {"users": [{"id": 1, "name": "A"}]}  |
|                                           |
|===== TCP close (FIN-ACK-FIN-ACK) =====|
```

HTTP не знает о TCP, TCP не знает о HTTP. TCP — труба, HTTP пишет текст в эту трубу.

1.4. HTTP/1.1, HTTP/2, HTTP/3

HTTP/1.1 (1997):

- Persistent Connections (Keep-Alive) — несколько запросов в одном TCP-соединении.
- Pipelining — клиент отправляет несколько запросов без ожидания ответа, но сервер отвечает строго по порядку.
- **Head-of-Line Blocking (HoL):** тяжелый запрос блокирует легкие.

HTTP/2 (2015):

- Бинарное фреймирование (HEADERS, DATA, SETTINGS, WINDOW_UPDATE).
- **Мультиплексирование:** одно TCP-соединение, множество потоков (streams) с разными ID. Медленный поток не блокирует другие на уровне HTTP.
- HPACK — сжатие заголовков.
- Но: все потоки делят одно TCP-соединение. Потеря одного TCP-пакета блокирует все потоки (HoL на транспортном уровне).

HTTP/3 (2022) + QUIC:

- Работает поверх UDP (в userspace).
- Независимая нумерация пакетов для каждого потока: потеря в потоке 1 не блокирует поток 3.
- 0-RTT / 1-RTT (вместо 2-RTT в TCP+TLS).
- Connection Migration — смена IP не разрывает соединение.

1.5. WebSocket

HTTP — запрос-ответ, клиент всегда иницирует. WebSocket — одно TCP-соединение, полный дуплекс.

Handshake (начинается как HTTP):

```
In [3]:  Клиент:
        GET /chat HTTP/1.1
        Host: example.com
        Upgrade: websocket
        Connection: Upgrade
        Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
        Sec-WebSocket-Version: 13

        Сервер:
        HTTP/1.1 101 Switching Protocols
        Upgrade: websocket
        Connection: Upgrade
        Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+x0o=
```

После 101 — бинарный режим WebSocket. Фреймы: текст (0x1), бинарные (0x2), close (0x8), ping (0x9), pong (0xA).

WebSocket vs SSE:

Критерий	WebSocket	SSE
Направление	Дуплекс	Сервер -> клиент
Протокол	TCP с Upgrade	HTTP/1.1 или HTTP/2
Переподключение	Ручное	Автоматическое
Бинарные данные	Нативно	Base64
Прокси	Могут блокировать	Работает через обычный HTTP

- **SSE** — сервер пушит (логи, прогресс, цены).
- **WebSocket** — двусторонний обмен (чат, real-time ML-инференс, аудио-стриминг).

1.6. Асинхронные сетевые библиотеки

```
In [4]: import asyncio

# asyncio низкого уровня: StreamReader, StreamWriter
async def tcp_echo_client(message: str):
    reader, writer = await asyncio.open_connection('127.0.0.1', 8888)
    print(f'Отправляю: {message}')
    writer.write(message.encode())
    await writer.drain()

    data = await reader.read(100)
    print(f'Получил: {data.decode()}')

    writer.close()
    await writer.wait_closed()

# asyncio.run(tcp_echo_client('Hello World'))
```

```
In [5]: # aiohttp: клиент с пулом соединений
import aiohttp
import asyncio

async def fetch_aiohttp():
    async with aiohttp.ClientSession() as session:
        async with session.get('https://httpbin.org/get') as response:
            print(f"Status: {response.status}")
            data = await response.json()
            print(data.keys())

asyncio.run(fetch_aiohttp())
```

```
In [6]: # httpx: единый API для sync/async + HTTP/2
import httpx
import asyncio

async def fetch_httpx():
    async with httpx.AsyncClient(http2=True) as client:
        response = await client.get('https://httpbin.org/get')
```

```
print(f"Status: {response.status_code}")
print(response.json().keys())

asyncio.run(fetch_httpx())
```

Золотое правило: никогда не вызывайте `requests.get()` внутри `async def` — это блокирует event loop. Используйте `httpx.AsyncClient` или `aiohttp`.

Часть 2. FastAPI: архитектура и жизненный цикл

2.1. Философия: Starlette + Pydantic

Компонент	Отвечает за	Аналогия
Starlette	ASGI-инфраструктура: роутинг, middleware, lifespan, Request/Response, WebSocket	Двигатель и шасси
Pydantic	Валидация, сериализация, генерация схем	Система безопасности и диагностики

FastAPI добавляет: автовалидацию через типы, автодокументацию Swagger UI / ReDoc, Dependency Injection (`Depends`), нативную поддержку `async / await`.

Type-driven development: сначала типы, потом код. Функция не начинает выполняться, пока контракт не подтвержден.

2.2. ASGI: от WSGI к асинхронности

WSGI (синхронный):

```
In [7]: def application(environ, start_response):
        status = '200 OK'
        headers = [('Content-Type', 'text/plain')]
        start_response(status, headers)
        return [b'Hello World']
```

Проблема: `application` — синхронная функция. Не может `await` чтения тела запроса или записи ответа. Worker-процесс блокируется.

ASGI (асинхронный):

```
In [8]: async def application(scope, receive, send):
    assert scope['type'] == 'http'

    body = b''
    while True:
        message = await receive()
        if message['type'] == 'http.request':
            body += message.get('body', b'')
            if not message.get('more_body', False):
                break

        await send({
            'type': 'http.response.start',
            'status': 200,
            'headers': [(b'content-type', b'text/plain')],
        })
        await send({
            'type': 'http.response.body',
            'body': b'Hello, ASGI!',
        })
```

Параметр	Назначение
<code>scope</code>	Контекст соединения: тип, метод, путь, заголовки. Живет все время соединения.
<code>receive</code>	Получение событий от клиента (тело запроса, WebSocket-сообщения).
<code>send</code>	Отправка событий клиенту (старт ответа, тело, закрытие).

ASGI-серверы:

Сервер	Особенности
Uvicorn	Быстрый, на <code>uvloop</code> (Cython), <code>httptools</code> (C). Стандарт для FastAPI.
Hypercorn	HTTP/2, HTTP/3. Написан на чистом Python, медленнее.
Daphne	Первый ASGI-сервер, для Django Channels.

```
In [9]: uvicorn main:app --host 0.0.0.0 --port 8000 --workers 4
```

2.3. Маршрутизация и обработчики

```
In [10]: from fastapi import FastAPI, APIRouter, Query, Path, Header, Cookie, File, Upload
File, Form
from pydantic import BaseModel, Field

app = FastAPI()

# --- APIRouter: модульность ---
router = APIRouter(prefix="/users", tags=["users"])

@router.get("/")
async def list_users():
    return [{"id": 1, "name": "Alice"}]

@router.get("/{user_id}")
async def get_user(user_id: int):
    return {"id": user_id, "name": "Alice"}

app.include_router(router)

# --- Path parameters с типизацией ---
@app.get("/items/{item_id}")
async def read_item(item_id: int):
    return {"item_id": item_id}

@app.get("/files/{file_path:path}")
async def read_file(file_path: str):
    return {"file_path": file_path}

# --- Query parameters с валидацией ---
@app.get("/search/")
async def search_items(
    q: str | None = Query(default=None, min_length=3, max_length=50),
    skip: int = Query(default=0, ge=0),
    limit: int = Query(default=10, ge=1, le=100),
):
    return {"q": q, "skip": skip, "limit": limit}

# --- Request Body: Pydantic модели ---
class Image(BaseModel):
    url: str
    name: str

class Item(BaseModel):
```



```

name: str = Field(min_length=1, max_length=100)
description: str | None = None
price: float = Field(gt=0)
tax: float | None = None
images: list[Image] = []

@app.post("/items/")
async def create_item(item: Item):
    return item

# --- Заголовки, куки, файлы, формы ---
@app.get("/headers/")
async def read_headers(user_agent: str | None = Header(default=None)):
    return {"User-Agent": user_agent}

@app.get("/cookies/")
async def read_cookies(session_id: str | None = Cookie(default=None)):
    return {"session_id": session_id}

@app.post("/upload/")
async def upload_file(file: UploadFile = File(...)):
    content = await file.read()
    return {"filename": file.filename, "size": len(content)}

@app.post("/login/")
async def login(username: str = Form(...), password: str = Form(...)):
    return {"username": username}

```

2.4. Pydantic V2: модели данных

Pydantic V2 (2023+) переписал ядро на Rust (`pydantic-core`). Преимущества: нет GIL, zero-сору парсинг, компиляция в машинный код. Валидация в 5-20 раз быстрее V1.

```

In [11]: from pydantic import BaseModel, Field, ConfigDict, field_validator, model_validator
or

class User(BaseModel):
    model_config = ConfigDict(strict=False)

    id: int
    name: str = Field(min_length=1)
    email: str
    age: int = Field(ge=0, le=150)

```

```

@field_validator('email')
@classmethod
def validate_email(cls, v: str) -> str:
    if '@' not in v:
        raise ValueError('Invalid email')
    return v.lower()

# Сериализация
user = User(id="42", name="Alice", email="Alice@Example.COM", age=30)
print(user.model_dump())          # dict
print(user.model_dump_json())     # JSON-строка
print(User.model_validate({"id": 42, "name": "Bob", "email": "b@b.com", "age": 25})) # из dict
print(User.model_validate_json('{"id": 42, "name": "Bob", "email": "b@b.com", "age": 25}')) # из JSON

# JSON Schema
print(User.model_json_schema())

```

```

In [12]: # Model validator
from pydantic import BaseModel, model_validator

class Rectangle(BaseModel):
    width: float
    height: float

    @model_validator(mode='after')
    def check_dimensions(self):
        if self.width <= 0 or self.height <= 0:
            raise ValueError('Dimensions must be positive')
        return self

rect = Rectangle(width=10, height=5)
print(rect)

# rect = Rectangle(width=-1, height=5) # ValidationError

```

2.5. Ответы клиенту

```

In [13]: from fastapi import FastAPI
from fastapi.responses import Response, JSONResponse, HTMLResponse, StreamingResponse, FileResponse
import asyncio

```

```

app = FastAPI()

# JSONResponse (по умолчанию)
@app.get("/json/")
async def get_json():
    return {"message": "hello"}

# HTMLResponse
@app.get("/html/", response_class=HTMLResponse)
async def get_html():
    return "<h1>Hello</h1>"

# StreamingResponse – критично для ML (токены по одному)
async def token_generator():
    tokens = ["Hello", " world", " from", " ML", "!"]
    for token in tokens:
        yield token
        await asyncio.sleep(0.1)

@app.get("/stream/")
async def stream_tokens():
    return StreamingResponse(token_generator(), media_type="text/plain")

# FileResponse – zero-copy через sendfile()
# @app.get("/download/{filename}")
# async def download_file(filename: str):
#     return FileResponse(path=f"/storage/{filename}", filename=filename)

```

2.6. Обработка ошибок

```

In [14]: from fastapi import FastAPI, HTTPException, Request
        from fastapi.responses import JSONResponse

app = FastAPI()

# HTTPException – стандартный механизм
@app.get("/items/{item_id}")
async def read_item(item_id: int):
    if item_id < 0:
        raise HTTPException(status_code=400, detail="Item ID must be positive")
    return {"item_id": item_id}

# --- Кастомные исключения и обработчики ---

```

```

class AppException(Exception):
    def __init__(self, message: str, status_code: int = 500):
        self.message = message
        self.status_code = status_code
        super().__init__(self.message)

class NotFoundError(AppException):
    def __init__(self, resource: str, resource_id: str):
        super().__init__(f"{resource} with id={resource_id} not found", 404)

class ValidationError(AppException):
    def __init__(self, field: str, reason: str):
        super().__init__(f"Validation failed for {field}: {reason}", 422)

@app.exception_handler(AppException)
async def app_exception_handler(request: Request, exc: AppException):
    return JSONResponse(
        status_code=exc.status_code,
        content={"error": exc.message, "type": exc.__class__.__name__}
    )

@app.exception_handler(Exception)
async def global_exception_handler(request: Request, exc: Exception):
    return JSONResponse(
        status_code=500,
        content={"error": "Internal server error"}
    )

# Использование в endpoint
@app.get("/users/{user_id}")
async def get_user(user_id: str):
    # Имитация: пользователь не найден
    raise NotFoundError("User", user_id)

```

2.7. Middleware

BaseHTTPMiddleware — простой, но опасен для streaming (буферизует весь ответ).

```

In [15]: from fastapi import FastAPI, Request
        from starlette.middleware.base import BaseHTTPMiddleware
        import time

        app = FastAPI()

        class TimingMiddleware(BaseHTTPMiddleware):

```

```

    async def dispatch(self, request: Request, call_next):
        start = time.perf_counter()
        response = await call_next(request)
        elapsed = time.perf_counter() - start
        response.headers["X-Process-Time"] = str(elapsed)
        return response

app.add_middleware(TimingMiddleware)

```

Чистая ASGI-middleware — полный контроль, нет накладных расходов, поддерживает streaming.

```

In [16]: class ASGITimingMiddleware:
        def __init__(self, app):
            self.app = app

        async def __call__(self, scope, receive, send):
            if scope["type"] != "http":
                await self.app(scope, receive, send)
                return

            start = time.perf_counter()

            async def wrapped_send(message):
                if message["type"] == "http.response.start":
                    elapsed = time.perf_counter() - start
                    headers = message.get("headers", [])
                    headers.append((b"x-process-time", str(elapsed).encode()))
                    message["headers"] = headers
                await send(message)

            await self.app(scope, receive, wrapped_send)

# app.add_middleware(ASGITimingMiddleware) # через custom middleware

```

Порядок выполнения (луковая модель):

```

In [17]: # Request: C -> B -> A -> Endpoint
        # Response: Endpoint -> A -> B -> C
        app.add_middleware(MiddlewareA)
        app.add_middleware(MiddlewareB)
        app.add_middleware(MiddlewareC)

```

2.8. Lifespan: жизненный цикл приложения

```
In [18]: from contextlib import asynccontextmanager
from fastapi import FastAPI

ml_model = None

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup
    global ml_model
    ml_model = {"name": "ResNet50", "loaded": True} # имитация загрузки модели
    app.state.model = ml_model
    print("Модель загружена")

    yield # приложение работает

    # Shutdown
    print("Модель освобождена")

app = FastAPI(lifespan=lifespan)

@app.get("/predict/")
async def predict():
    model = app.state.model
    return {"model": model["name"], "prediction": [0.1, 0.9]}

# AsyncExitStack для нескольких ресурсов
from contextlib import AsyncExitStack

@asynccontextmanager
async def lifespan_multi(app: FastAPI):
    async with AsyncExitStack() as stack:
        # db = await create_db_pool()
        # await stack.enter_async_context(db)
        # redis = await create_redis_client()
        # await stack.enter_async_context(redis)
        app.state.config = {"db": "connected", "redis": "connected"}
        yield
    # ресурсы закрываются автоматически
```

Graceful shutdown: Uvicorn получает SIGTERM -> прекращает прием новых соединений -> ждет завершения текущих запросов (timeout 30s) -> lifespan shutdown -> завершает процесс.

```
In [19]: uvicorn main:app --timeout-graceful-shutdown 60
```

Задания для самостоятельной работы

Задание 1. TCP-клиент на чистом asyncio

Напишите TCP-клиент, который подключается к echo-серверу (можно использовать `nc -l 8888` для тестирования), отправляет сообщение и выводит ответ. Используйте `asyncio.open_connection`.

```
In [20]: import asyncio

async def tcp_client(host: str, port: str, message: str) -> str:

    reader, writer = await asyncio.open_connection(host, port)

    print(f"Отправляю: {message}")
    writer.write(message.encode())
    await writer.drain()

    data = await reader.read(1024)
    response = data.decode()
    print(f"Получил: {response}")

    writer.close()
    await writer.wait_closed()

    return response
```

```
In [21]: import asyncio

async def tcp_client(host: str, port: int, message: str) -> str:
    reader, writer = await asyncio.open_connection(host, port)

    print(f"Отправляю: {message}")
    writer.write(message.encode())
    await writer.drain()

    data = await reader.read(1024)
    response = data.decode()
```

```
print(f"Получил: {response}")

writer.close()
await writer.wait_closed()
return response

# Для тестирования запустите в терминале: nc -l 8888
# asyncio.run(tcp_client('127.0.0.1', 8888, 'Hello, TCP!'))
```

Задание 2. TCP-echo сервер на asyncio

Напишите TCP-сервер, который принимает соединения, читает данные и отправляет их обратно (echo). Используйте `asyncio.start_server`.

```
In [22]: import asyncio

async def handle_client(reader: asyncio.StreamReader, writer: asyncio.StreamWriter):
    addr = writer.get_extra_info('peername')
    print(f"Подключение от {addr}")

    while True:
        data = await reader.read(100)
        if not data:
            break
        message = data.decode()
        print(f"Получено от {addr}: {message}")
        writer.write(data)
        await writer.drain()

    print(f"Отключение {addr}")
    writer.close()
    await writer.wait_closed()

async def run_server(host: str = '127.0.0.1', port: int = 8888):
    server = await asyncio.start_server(handle_client, host, port)
    print(f"Сервер запущен на {host}:{port}")

    async with server:
        await server.serve_forever()

# asyncio.run(run_server())
```


Задание 3. Конкурентные HTTP-запросы через aiohttp

Напишите функцию `fetch_all(urls)`, которая конкурентно делает GET-запросы к списку URL через `aiohttp`. Верните список статус-кодов в том же порядке, что и URL. Используйте `asyncio.gather` и `ClientSession`.

```
In [23]: import aiohttp
import asyncio

async def fetch_one(session: aiohttp.ClientSession, url: str) -> int:
    async with session.get(url) as response:
        return response.status

async def fetch_all(urls: list[str]) -> list[int]:
    async with aiohttp.ClientSession() as session:
        tasks = [fetch_one(session, url) for url in urls]
        return await asyncio.gather(*tasks)

async def main():
    urls = [
        "https://httpbin.org/get",
        "https://httpbin.org/ip",
        "https://httpbin.org/user-agent",
    ]
    statuses = await fetch_all(urls)
    for url, status in zip(urls, statuses):
        print(f"{url}: {status}")

asyncio.run(main())
```

Задание 4. Ограничение конкурентных запросов через семафор

Модифицируйте задание 3: добавьте `asyncio.Semaphore(3)`, чтобы одновременно выполнялось не более 3 запросов. Добавьте `print` при старте и завершении каждого запроса.

```
In [24]: import aiohttp
import asyncio

semaphore = asyncio.Semaphore(3)

async def fetch_limited(session: aiohttp.ClientSession, url: str) -> int:
    async with semaphore:
```

```

    print(f"  START {url}")
    async with session.get(url) as response:
        status = response.status
        print(f"  DONE  {url} -> {status}")
        return status

async def fetch_all_limited(urls: list[str]) -> list[int]:
    async with aiohttp.ClientSession() as session:
        tasks = [fetch_limited(session, url) for url in urls]
        return await asyncio.gather(*tasks)

async def main():
    urls = [f"https://httpbin.org/delay/{i}" for i in [1, 1, 1, 1, 1]]
    statuses = await fetch_all_limited(urls)
    print(f"\nBcero: {len(statuses)}")

asyncio.run(main())

```

Задание 5. "Голое" ASGI-приложение

Напишите минимальное ASGI-приложение, которое:

1. Читает тело POST-запроса.
2. Парсит его как JSON (можно использовать `json.loads`).
3. Возвращает JSON с полем `echo` , содержащим полученные данные.

```

In [25]: import json

async def echo_app(scope, receive, send):
    assert scope['type'] == 'http'

    # Читаем тело запроса
    body = b''
    while True:
        message = await receive()
        if message['type'] == 'http.request':
            body += message.get('body', b'')
            if not message.get('more_body', False):
                break

    # Парсим JSON
    try:
        data = json.loads(body.decode()) if body else {}
    except json.JSONDecodeError:

```

```

data = {"error": "Invalid JSON"}

# Формируем ответ
response_body = json.dumps({"echo": data}).encode()

await send({
    'type': 'http.response.start',
    'status': 200,
    'headers': [(b'content-type', b'application/json')],
})
await send({
    'type': 'http.response.body',
    'body': response_body,
})

# Запуск: uvicorn seminar_34:echo_app --port 8000
# Тест: curl -X POST http://localhost:8000/ -H "Content-Type: application/json" -d '{"hello":"world"}'

```

Задание 6. FastAPI с APIRouter

Создайте приложение FastAPI с двумя роутерами: `users` (prefix="/users") и `items` (prefix="/items"). Каждый роутер должен иметь минимум 2 endpoint'a: `GET /` (список) и `GET /{id}` (детали). Подключите роутеры в main-приложении.

```

In [26]: from fastapi import FastAPI, APIRouter

# --- Роутер users ---
users_router = APIRouter(prefix="/users", tags=["users"])

@users_router.get("/")
async def list_users():
    return [{"id": 1, "name": "Alice"}, {"id": 2, "name": "Bob"}]

@users_router.get("/{user_id}")
async def get_user(user_id: int):
    return {"id": user_id, "name": f"User {user_id}"}

# --- Роутер items ---
items_router = APIRouter(prefix="/items", tags=["items"])

@items_router.get("/")
async def list_items():
    return [{"id": 1, "title": "Laptop"}, {"id": 2, "title": "Phone"}]

```

```

@items_router.get("/{item_id}")
async def get_item(item_id: int):
    return {"id": item_id, "title": f"Item {item_id}"}

# --- Main app ---
app = FastAPI(title="Shop API")
app.include_router(users_router)
app.include_router(items_router)

# uvicorn seminar_34:app --reload
# Документация: http://localhost:8000/docs

```

Задание 7. Pydantic-модель с вложенной валидацией

Создайте модели `Address` (city, country) и `Person` (name, age, addresses: list[Address]).
Добавьте:

- `field_validator` для email (если добавите поле email).
- `model_validator` для проверки, что возраст ≥ 0 .
- Продемонстрируйте `model_dump()`, `model_dump_json()`, `model_json_schema()`.

```

In [27]: from pydantic import BaseModel, Field, field_validator, model_validator
        from typing import List

        class Address(BaseModel):
            city: str = Field(min_length=1)
            country: str = Field(min_length=1)

        class Person(BaseModel):
            name: str = Field(min_length=1, max_length=100)
            age: int = Field(ge=0, le=150)
            email: str
            addresses: List[Address] = []

        @field_validator('email')
        @classmethod
        def validate_email(cls, v: str) -> str:
            if '@' not in v:
                raise ValueError('Invalid email format')
            return v.lower()

        @model_validator(mode='after')

```

```

def check_age(self):
    if self.age < 0:
        raise ValueError('Age cannot be negative')
    return self

# Создание и сериализация
person = Person(
    name="Alice Johnson",
    age=30,
    email="Alice@Example.COM",
    addresses=[
        Address(city="New York", country="USA"),
        Address(city="London", country="UK")
    ]
)

print("=== model_dump ===")
print(person.model_dump())

print("\n=== model_dump_json ===")
print(person.model_dump_json(indent=2))

print("\n=== model_json_schema ===")
import json
print(json.dumps(Person.model_json_schema(), indent=2))

```

Задание 8. StreamingResponse для генерации токенов

Напишите endpoint `/stream/`, который возвращает `StreamingResponse`. Генератор должен выдавать "токены" (слова) с задержкой 0.2 секунды между ними. Клиент получает данные по мере генерации.

```

In [28]: from fastapi import FastAPI
from fastapi.responses import StreamingResponse
import asyncio

app = FastAPI()

async def token_generator():
    tokens = ["The", " quick", " brown", " fox", " jumps", " over", " the", " laz",
y", " dog", "."]
    for token in tokens:
        await asyncio.sleep(0.2)
        yield token

```

```
@app.get("/stream/")
async def stream_tokens():
    return StreamingResponse(token_generator(), media_type="text/plain")

# uvicorn seminar_34:app --port 8000
# Тест: curl http://localhost:8000/stream/ (должен показывать токены по мере генерации)
```

Задание 9. Кастомные Exception Handlers

Создайте иерархию исключений (`AppException` , `NotFoundError` , `ValidationError`) и зарегистрируйте для них обработчики. Endpoint `/users/{user_id}` должен выбрасывать `NotFoundError` , если `user_id` не существует в "базе данных" (словаре). Endpoint `/validate/` должен выбрасывать `ValidationError` при некорректных данных.

```
In [29]: from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse

app = FastAPI()

# --- Иерархия исключений ---
class AppException(Exception):
    def __init__(self, message: str, status_code: int = 500):
        self.message = message
        self.status_code = status_code
        super().__init__(self.message)

class NotFoundError(AppException):
    def __init__(self, resource: str, resource_id: str):
        super().__init__(f"{resource} with id={resource_id} not found", 404)

class ValidationError(AppException):
    def __init__(self, field: str, reason: str):
        super().__init__(f"Validation failed for {field}: {reason}", 422)

# --- Обработчики ---
@app.exception_handler(AppException)
async def app_exception_handler(request: Request, exc: AppException):
    return JSONResponse(
        status_code=exc.status_code,
        content={"error": exc.message, "type": exc.__class__.__name__}
    )
```

```

@app.exception_handler(Exception)
async def global_exception_handler(request: Request, exc: Exception):
    return JSONResponse(
        status_code=500,
        content={"error": "Internal server error", "detail": str(exc)}
    )

# --- "База данных" ---
USERS_DB = {
    "1": {"id": "1", "name": "Alice"},
    "2": {"id": "2", "name": "Bob"},
}

# --- Endpoint'ы ---
@app.get("/users/{user_id}")
async def get_user(user_id: str):
    if user_id not in USERS_DB:
        raise NotFoundError("User", user_id)
    return USERS_DB[user_id]

@app.post("/validate/")
async def validate_data(data: dict):
    if "email" not in data:
        raise ValidationError("email", "field is required")
    if "@" not in data["email"]:
        raise ValidationError("email", "invalid format")
    return {"valid": True}

@app.get("/error/")
async def trigger_error():
    raise ValueError("Unexpected error")

# uvicorn seminar_34:app --port 8000
# Тесты:
# curl http://localhost:8000/users/99
# curl -X POST http://localhost:8000/validate/ -H "Content-Type: application/json" -d '{"name": "test"}'
# curl http://localhost:8000/error/

```

Задание 10. Чистая ASGI-middleware для логирования

Напишите ASGI-middleware `LoggingMiddleware`, которая логирует: метод, путь, статус ответа, время обработки. Используйте перехват `send` для получения статуса. Не используйте `BaseHTTPMiddleware`.

```
In [30]: import time
from fastapi import FastAPI

app = FastAPI()

class LoggingMiddleware:
    def __init__(self, app):
        self.app = app

    async def __call__(self, scope, receive, send):
        if scope["type"] != "http":
            await self.app(scope, receive, send)
            return

        method = scope.get("method", "UNKNOWN")
        path = scope.get("path", "UNKNOWN")
        start = time.perf_counter()

        async def wrapped_send(message):
            if message["type"] == "http.response.start":
                status = message.get("status", 0)
                elapsed = time.perf_counter() - start
                print(f"[{method}] {path} -> {status} ({elapsed:.4f}s)")
            await send(message)

        await self.app(scope, receive, wrapped_send)

# Добавляем middleware
app.add_middleware(LoggingMiddleware)

@app.get("/fast/")
async def fast_endpoint():
    return {"status": "ok"}

@app.get("/slow/")
async def slow_endpoint():
    import asyncio
    await asyncio.sleep(0.5)
    return {"status": "ok"}

# uvicorn seminar_34:app --port 8000
# curl http://localhost:8000/fast/
# curl http://localhost:8000/slow/
```


Задание 11. Lifespan с загрузкой ML-модели

Напишите FastAPI-приложение с `lifespan`, которое:

1. При старте "загружает" ML-модель (словарь с параметрами) в `app.state.model`.
2. Endpoint `/predict/` использует модель из `app.state`.
3. При остановке печатает сообщение об освобождении.

```
In [31]: from contextlib import asynccontextmanager
from fastapi import FastAPI
from pydantic import BaseModel
import asyncio

class PredictionInput(BaseModel):
    features: list[float]

# --- Lifespan ---
@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup: загрузка модели
    print("Loading ML model...")
    await asyncio.sleep(1) # имитация загрузки
    app.state.model = {
        "name": "RandomForestClassifier",
        "version": "1.0.0",
        "params": {"n_estimators": 100}
    }
    print("Model loaded successfully")

    yield # приложение работает

    # Shutdown: освобождение ресурсов
    print("Shutting down... Releasing model resources")
    app.state.model = None

app = FastAPI(lifespan=lifespan)

# --- Endpoint ---
@app.post("/predict/")
async def predict(input_data: PredictionInput):
    model = app.state.model
    if model is None:
        return {"error": "Model not loaded"}
```

```

# Имитация inference
features = input_data.features
prediction = sum(features) / len(features) if features else 0
return {
    "model": model["name"],
    "version": model["version"],
    "prediction": prediction,
    "input_features": features
}

@app.get("/model/")
async def get_model_info():
    return app.state.model or {"error": "No model loaded"}

# uvicorn seminar_34:app --port 8000
# curl -X POST http://localhost:8000/predict/ -H "Content-Type: application/json"
# -d '{"features":[1.0,2.0,3.0]}'
# curl http://localhost:8000/model/

```

Задание 12. Полноценный endpoint с валидацией всех частей запроса

Создайте endpoint `POST /orders/`, который принимает:

- Path: нет (только `/orders/`).
- Query: `?urgent=true` (bool, по умолчанию false).
- Body: Pydantic-модель `Order` (customer_name, items: list[Item], где Item = name + quantity + price).
- Header: `X-Request-ID` (str, опционально).

Верните подтверждение заказа с общей суммой.

```

In [32]: from fastapi import FastAPI, Query, Header
        from pydantic import BaseModel, Field
        from typing import List, Optional

        app = FastAPI()

        class Item(BaseModel):
            name: str = Field(min_length=1)
            quantity: int = Field(ge=1)
            price: float = Field(gt=0)

        class Order(BaseModel):

```

```

customer_name: str = Field(min_length=1, max_length=100)
items: List[Item]

@app.post("/orders/")
async def create_order(
    order: Order,
    urgent: bool = Query(default=False),
    request_id: Optional[str] = Header(default=None, alias="X-Request-ID")
):
    total = sum(item.quantity * item.price for item in order.items)
    return {
        "request_id": request_id,
        "urgent": urgent,
        "customer": order.customer_name,
        "items_count": len(order.items),
        "total": round(total, 2),
        "status": "confirmed"
    }

# uvicorn seminar_34:app --port 8000
# curl -X POST "http://localhost:8000/orders/?urgent=true" \
#   -H "Content-Type: application/json" \
#   -H "X-Request-ID: req-123" \
#   -d '{"customer_name": "Alice", "items": [{"name": "Laptop", "quantity": 1, "price": 99.99}, {"name": "Mouse", "quantity": 2, "price": 29.99}]}'

```

Итоговая сводка

Концепция	Модуль	Ключевой API / Паттерн
OSI/TCP/IP	3	7/4 уровня, инкапсуляция
TCP handshake	3	SYN-SYN/ACK-ACK, 1 RTT
TCP AIMD	3	Slow Start, Congestion Avoidance, cwnd
HTTP/1.1	3	Keep-Alive, Pipelining, HoL blocking
HTTP/2	3	Бинарные фреймы, мультиплексирование, HPACK
HTTP/3/QUIC	3	UDP, независимые потоки, 0-RTT
WebSocket	3	Upgrade: websocket, 101, фреймы, ping/pong
SSE	3	Server-Sent Events, односторонний push

Концепция	Модуль	Ключевой API / Паттерн
aiohttp/httpx	3	Асинхронные клиенты, пулы соединений
ASGI	4	scope/receive/send, async callable
Uvicorn	4	uvloop + httptools
APIRouter	4	prefix, tags, include_router
Path/Query параметры	4	Path(...), Query(ge=, le=)
Pydantic V2	4	BaseModel, Field, model_dump, model_validate_json
Валидаторы	4	@field_validator, @model_validator
Response типы	4	JSONResponse, StreamingResponse, FileResponse
HTTPException	4	raise HTTPException(status_code=, detail=)
Exception Handlers	4	@app.exception_handler()
BaseHTTPMiddleware	4	Простой, но опасен для streaming
ASGI Middleware	4	call (scope, receive, send)
Lifespan	4	@asynccontextmanager, startup/shutdown
app.state	4	Глобальное состояние приложения

Чек-лист для самопроверки

- ☐ Я могу объяснить разницу между HTTP/1.1, HTTP/2 и HTTP/3.
- ☐ Я понимаю, что такое Head-of-Line Blocking и на каких уровнях он проявляется.
- ☐ Я знаю, когда использовать WebSocket, а когда — SSE.
- ☐ Я могу написать "голое" ASGI-приложение с нуля.
- ☐ Я понимаю разницу между WSGI и ASGI.
- ☐ Я умею создавать APIRouter и подключать их к приложению.
- ☐ Я могу писать Pydantic-модели с field_validator и model_validator.
- ☐ Я знаю, когда использовать BaseHTTPMiddleware, а когда — чистую ASGI-middleware.
- ☐ Я умею управлять жизненным циклом приложения через lifespan.
- ☐ Я понимаю, как работает graceful shutdown в Uvicorn.